



EAST WEST UNIVERSITY

Department of Computer Science and Engineering

Semester: Fall 2023

Course Code: CSE 207

Course Title: Data Structures

Section:7

Submitted to

Md. Manowarul Islam

Adjunct Faculty, Department of CSE

Submitted By

Swarna Rani Dey

2022-1-60-340

Submitted On

13.12.2023

Exercise:1

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct Node {  
    int data;  
    struct Node *next;  
} Node;
```

```
struct Queue {  
    struct Node *front;  
    struct Node *rear;  
};
```

```
void enqueue(struct Queue *queue, int data) {  
    struct Node *newNode = (struct Node *)malloc(sizeof(struct  
Node));
```

```
    newNode->data = data;  
    newNode->next = NULL;
```

```
    if (queue->front == NULL && queue->rear == NULL) {  
        queue->front = newNode;
```

```
    queue->rear = newNode;
} else {
    queue->rear->next = newNode;
    queue->rear = newNode;
}
}
```

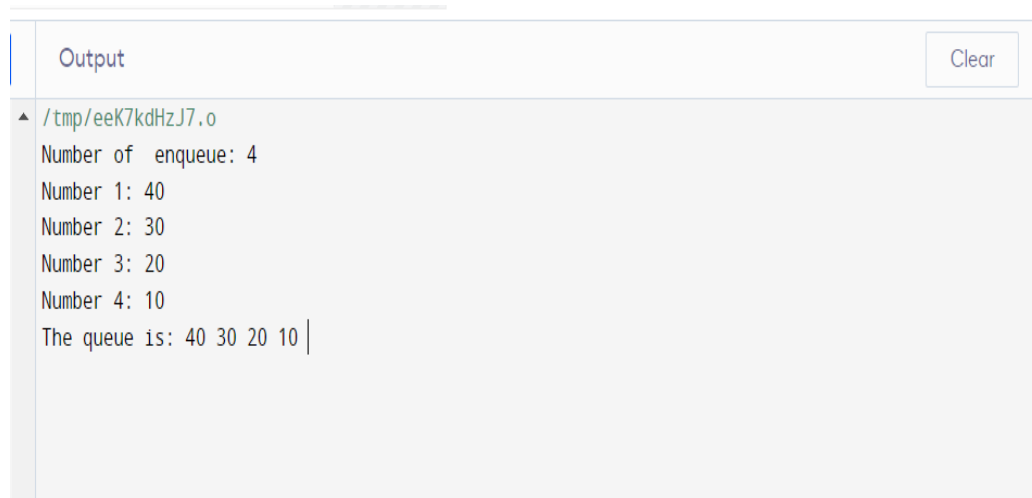
```
int main() {
    struct Queue *queue = (struct Queue *)malloc(sizeof(struct
Queue));
    queue->front = NULL;
    queue->rear = NULL;
```

```
    int n;
    printf("Number of enqueue: ");
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {
        int data;
        printf("Number %d: ", i + 1);
        scanf("%d", &data);
        enqueue(queue, data);
    }
```

```
printf("The queue is: ");  
struct Node *currentNode = queue->front;  
while (currentNode != NULL) {  
  
    printf("%d ", currentNode->data);  
    currentNode = currentNode->next;  
}  
  
return 0;  
}
```

OUTPUT



```
Output Clear  
^ /tmp/eeK7kdHzJ7.o  
Number of enqueue: 4  
Number 1: 40  
Number 2: 30  
Number 3: 20  
Number 4: 10  
The queue is: 40 30 20 10 |
```

Exercise:2

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
typedef struct
```

```
Node {
```

```
int data;
```

```
struct
```

```
Node *next;
```

```
} Node;
```

```
typedef struct
```

```
Queue {
```

```
struct
```

```
Node *front;
```

```
struct
```

```
Node *rear;  
} Queue;
```

```
void enqueue(Queue *q, int data) {  
    struct Node *newNode = malloc(sizeof(struct Node));
```

```
    newNode->data = data;  
    newNode->next = NULL;
```

```
    if (q->front == NULL) {  
        q->front = q->rear = newNode;  
    } else {  
        q->rear->next = newNode;  
        q->rear = newNode;  
    }  
}
```

```
int
```

```
dequeue(Queue *q)
```

```
{  
    if (q->front == NULL) {
```

```
    printf("Error: Queue is empty\n");  
    return -1;  
}
```

```
struct Node *temp = q->front;  
int data = temp->data;  
q->front = q->front->next;
```

```
if (q->front == NULL) {  
    q->rear = NULL;  
}
```

```
free(temp);  
return data;  
}
```

```
void printQueue(Queue *q) {  
    struct Node *temp = q->front;  
  
    while (temp != NULL) {  
        printf("%d ", temp->data);  
        temp = temp->next;  
    }  
}
```

```
printf("\n");  
}
```

```
int main() {  
    Queue *q = malloc(sizeof(Queue));  
    q->front = q->rear = NULL;  
  
    int n;  
    printf("Enter the number of elements: ");  
    scanf("%d", &n);  
  
    for (int i = 0; i < n; i++) {  
        int data;  
        printf("Enter element %d: ", i + 1);  
        scanf("%d", &data);  
  
        enqueue(q, data);  
    }  
  
    int deletedData = dequeue(q);  
    printf("Deleted data: %d\n", deletedData);
```



```
printf("Updated queue: ");  
printQueue(q);  
  
free(q);  
  
return 0;  
}
```

OUTPUT



```
Output Clear  
^ /tmp/vd2DFD3jkx.o  
Enter the number of elements: 5  
Enter element 1: 5  
Enter element 2: 6  
Enter element 3: 7  
Enter element 4: 8  
Enter element 5: 9  
Deleted data: 5  
Updated queue: 6 7 8 9
```

Exercise:3

```
#include <stdio.h>  
  
#include <stdlib.h>  
  
typedef struct Node {  
    int data;  
    struct Node *next;
```

```
} Node;
```

```
typedef struct Queue {  
    struct Node *head;  
    struct Node *tail;  
} Queue;
```

```
Queue *createQueue() {  
    Queue *queue = (Queue *)malloc(sizeof(Queue));  
    queue->head = NULL;  
    queue->tail = NULL;  
    return queue;  
}
```

```
void enqueue(Queue *queue, int data) {  
    Node *newNode = (Node *)malloc(sizeof(Node));  
    newNode->data = data;  
    newNode->next = NULL;  
  
    if (queue->head == NULL) {  
        queue->head = newNode;  
        queue->tail = newNode;  
    } else {
```

```

    queue->tail->next = newNode;
    queue->tail = newNode;
}
}

int dequeue(Queue *queue) {
    if (queue->head == NULL) {
        return -1;
    }

    int data = queue->head->data;
    Node *temp = queue->head;
    queue->head = queue->head->next;
    free(temp);

    if (queue->head == NULL) {
        queue->tail = NULL;
    }
    return data;
}

int isEmpty(Queue *queue) {
    return queue->head == NULL;
}

```

```
void printQueue(Queue *queue) {
    Node *temp = queue->head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    Queue *queue = createQueue();

    int n;
    printf("Enter the number of elements to enqueue: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        int data;
        printf("Enter element %d: ", i + 1);
        scanf("%d", &data);
        enqueue(queue, data);
    }
```

```
printf("The queue is: ");  
printQueue(queue);
```

```
printf("\nEnter the number of elements to dequeue: ");  
scanf("%d", &n);
```

```
for (int i = 0; i < n; i++) {  
    int data = dequeue(queue);  
    if (data == -1) {  
        printf("The queue is empty\n");  
        break;  
    } else {  
        printf("Dequeued element %d: %d\n", i + 1, data);  
    }  
}
```

```
printf("The queue is: ");  
printQueue(queue);
```

```
return 0;  
}
```

OUTPUT

```
Output Clear
^ /tmp/vd2DFD3jkx.o
Enter the number of elements to enqueue: 7
Enter element 1: 50
Enter element 2: 70
Enter element 3: 90
Enter element 4: 110
Enter element 5: 130
Enter element 6: 150
Enter element 7: 170
The queue is: 50 70 90 110 130 150 170

Enter the number of elements to dequeue: 2
Dequeued element 1: 50
Dequeued element 2: 70
The queue is: 90 110 130 150 170
|
```

Exercise:4

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct QueueNode {
    int data;
    struct QueueNode* next;
} QueueNode;
```

```
typedef struct Queue {
    QueueNode* front;
    QueueNode* rear;
    int size;
```

```
} Queue;
```

```
Queue* createQueue() {  
    Queue* queue = (Queue*)malloc(sizeof(Queue));  
    queue->front = queue->rear = NULL;  
    queue->size = 0;  
    return queue;  
}
```

```
int isEmpty(Queue* queue) {  
    return (queue->size == 0);  
}
```

```
void enqueue(Queue* queue, int data) {  
    QueueNode* newNode =  
    (QueueNode*)malloc(sizeof(QueueNode));  
  
    newNode->data = data;  
    newNode->next = NULL;  
  
    if (queue->rear == NULL) {  
        queue->front = queue->rear = newNode;  
    } else {
```

```
queue->rear->next = newNode;  
queue->rear = newNode;  
}
```

```
queue->size++;  
}
```

```
int dequeue(Queue* queue) {  
    if (isEmpty(queue)) {  
        printf("Queue is empty.\n");  
        exit(EXIT_FAILURE);  
    }
```

```
    int data = queue->front->data;  
    QueueNode* temp = queue->front;
```

```
    queue->front = queue->front->next;
```

```
    if (queue->front == NULL) {  
        queue->rear = NULL;  
    }
```

```
    free(temp);
```



```
queue->size--;
```

```
return data;
```

```
}
```

```
void copyQueue(Queue* source, Queue* destination) {
```

```
    QueueNode* current = source->front;
```

```
    while (current != NULL) {
```

```
        enqueue(destination, current->data);
```

```
        current = current->next;
```

```
    }
```

```
}
```

```
void displayQueue(Queue* queue) {
```

```
    QueueNode* current = queue->front;
```

```
    printf("Queue: ");
```

```
    while (current != NULL) {
```

```
        printf("%d ", current->data);
```

```
        current = current->next;
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
int main() {  
    Queue* sourceQueue = createQueue();  
    Queue* destinationQueue = createQueue();  
  
    int n, data;  
    printf("Enter the number of elements in the source queue: ");  
    scanf("%d", &n);  
  
    printf("Enter the elements of the source queue:\n");  
    for (int i = 0; i < n; ++i) {  
  
        scanf("%d", &data);  
        enqueue(sourceQueue, data);  
    }  
  
    copyQueue(sourceQueue, destinationQueue);  
  
    displayQueue(destinationQueue);  
    free(sourceQueue);  
    free(destinationQueue);  
  
    return 0;  
}
```

```
}
```

OUTPUT

```
Output Clear  
^ /tmp/vd2DFD3jkx.o  
Enter the number of elements in the source queue: 5  
Enter the elements of the source queue:  
1  
2  
3  
4  
5  
Queue: 1 2 3 4 5  
|
```

Exercise:5

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct Queue {  
    int front, rear, size;  
    unsigned capacity;  
    char* array;  
};
```

```
struct Queue* createQueue(unsigned capacity) {  
    struct Queue* queue = (struct Queue*)malloc(sizeof(struct  
Queue));  
  
    queue->capacity = capacity;  
    queue->front = queue->size = 0;  
    queue->rear = capacity - 1;  
    queue->array = (char*)malloc(queue->capacity *  
sizeof(char));  
    return queue;  
}
```

```
int isFull(struct Queue* queue) {  
    return (queue->size == queue->capacity);  
}
```

```
int isEmpty(struct Queue* queue) {  
    return (queue->size == 0);  
}
```

```
void enqueue(struct Queue* queue, char item) {  
    if (isFull(queue))
```

```
    return;
    queue->rear = (queue->rear + 1) % queue->capacity;
    queue->array[queue->rear] = item;
    queue->size++;
}
```

```
char dequeue(struct Queue* queue) {
    if (isEmpty(queue))
        return '\0';
    char item = queue->array[queue->front];
    queue->front = (queue->front + 1) % queue->capacity;
    queue->size--;
    return item;
}
```

```
void compressString(char* str) {
    struct Queue* queue = createQueue(strlen(str));

    for (int i = 0; str[i]; i++) {
        if (str[i] != ' ') {
            enqueue(queue, str[i]);
        }
    }
}
```

```
int i = 0;
while (!isEmpty(queue)) {
    str[i++] = dequeue(queue);
}
str[i] = '\0';

free(queue->array);
free(queue);
}

int main() {
    printf("Enter a string: ");
    char input[100];
    fgets(input, sizeof(input), stdin);

    compressString(input);

    printf("Compressed string: %s\n", input);

    return 0;
}
```

OUTPUT

```
Output Clear  
^ /tmp/TY2nBvugoE.o  
Enter a string: we are the students of CSE  
Compressed string: wearethestudentsofCSE
```

Exercise-6

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct Node {  
    int data;  
    struct Node *next;  
} Node;
```

```
typedef struct
```

```
Stack {
```

```
Node *top;  
} Stack;
```

```
typedef struct
```

```
Queue {  
    Node *front;  
    Node *rear;  
} Queue;
```

```
Node *createNode(int value) {  
    Node *newNode = (Node *)malloc(sizeof(Node));  
    newNode->data = value;  
    newNode->next = NULL;  
    return newNode;  
}
```

```
void
```

```
push(Stack *stack, int value)
```

```
{  
    Node *newNode = createNode(value);
```



```
newNode->next = stack->top;
stack->top = newNode;
}
```

```
int pop(Stack *stack) {
    if (stack->top == NULL) {
        return -1;
    }
}
```

```
int value = stack->top->data;
Node *temp = stack->top;
stack->top = stack->top->next;
free(temp);
return value;
}
```

```
void enqueue(Queue *queue, int value) {
    Node *newNode = createNode(value);
    if (queue->rear == NULL) {
        queue->front = queue->rear = newNode;
    } else {
        queue->rear->next = newNode;
        queue->rear = newNode;
    }
}
```

```
}  
}
```

```
void printQueue(Queue *queue) {  
    Node *current = queue->front;  
    while (current != NULL) {  
        printf("%d ", current->data);  
        current = current->next;  
    }  
    printf("\n");  
}
```

```
int main() {  
    Stack *stack = (Stack *)malloc(sizeof(Stack));  
    stack->top = NULL;  
  
    Queue *queue = (Queue *)malloc(sizeof(Queue));  
    queue->front = NULL;  
    queue->rear = NULL;  
  
    int n, value;  
    printf("Enter the total elements in the stack: ");  
    scanf("%d", &n);
```

```
for (int i = 0; i < n; i++) {  
    scanf("%d", &value);  
    push(stack, value);  
}  
  
while (stack->top != NULL) {  
    int poppedValue = pop(stack);  
    enqueue(queue, poppedValue);  
}  
  
printf("Queue created from the stack: ");  
printQueue(queue);  
  
free(stack);  
free(queue);  
  
return 0;  
}
```

OUTPUT

```
Output Clear
^ /tmp/gfKak1nU1s.o
Enter the total elements in the stack: 5
1
2
3
4
5
Queue created from the stack: 5 4 3 2 1
```

Exercise-7

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node* next;
};
```

```
struct Queue {
    struct Node* front;
    struct Node* rear;
};
```

```
void initializeQueue(struct Queue* q) {
    q->front = q->rear = NULL;
```

```
}
```

```
void enqueue(struct Queue* q, int data) {
```

```
    struct Node* newNode = (struct  
Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = data;
```

```
    newNode->next = NULL;
```

```
    if (q->rear == NULL) {
```

```
        q->front = q->rear = newNode;
```

```
        return;
```

```
    }
```

```
    q->rear->next = newNode;
```

```
    q->rear = newNode;
```

```
}
```

```
int dequeue(struct Queue* q) {
```

```
    if (q->front == NULL) {
```

```
        return -1;
```

```
    }
```

```
    struct Node* temp = q->front;
```

```
int data = temp->data;
```

```
q->front = q->front->next;
```

```
if (q->front == NULL) {  
    q->rear = NULL;  
}
```

```
free(temp);
```

```
return data;  
}
```

```
void deleteNegativeIntegers(struct Queue* q) {
```

```
    struct Queue tempQueue;
```

```
    initializeQueue(&tempQueue);
```

```
    while (q->front != NULL) {
```

```
        int element = dequeue(q);
```

```
        if (element >= 0) {
```

```
            enqueue(&tempQueue, element);
```

```
        }
```

```
}
```

```
while (tempQueue.front != NULL) {  
    enqueue(q, dequeue(&tempQueue));  
}
```

```
}
```

```
void displayQueue(struct Queue* q) {  
    struct Node* current = q->front;  
    while (current != NULL) {  
        printf("%d ", current->data);  
        current = current->next;  
    }  
    printf("\n");  
}
```

```
int main() {  
    struct Queue myQueue;  
    initializeQueue(&myQueue);
```

```
    int n;
```

```
    printf("Enter the number of elements in the queue: ");
```

```
scanf("%d", &n);
```

```
printf("Enter the elements of the queue:\n");
```

```
for (int i = 0; i < n; i++) {
```

```
    int element;
```

```
    scanf("%d", &element);
```

```
    enqueue(&myQueue, element);
```

```
}
```

```
deleteNegativeIntegers(&myQueue);
```

```
printf("Modified Queue after deleting negative integers: ");
```

```
displayQueue(&myQueue);
```

```
return 0;
```

```
}
```

OUTPUT

Output

Clear

▲ /tmp/08AwjLIPsx.o

Enter the number of elements in the queue: 5

Enter the elements of the queue:

1

2

-3

4

-5

Modified Queue after deleting negative integers: 1 2 4